

Neural networks for solving PDEs

From data-driven regression to PINNs, Deep Ritz, and hybrid losses

Contents

From data to PDE solvers

- Data-driven learning

- PDE-informed learning

Strong-form PINNs

- Formulation and collocation

- Quadrature, stability, and error

- Training and approximation limits

Network design and boundary constraints

- Regularity and architecture

- Boundary conditions

Variational and weak methods

- Deep Ritz methods

- Weak formulations

Hybrid PDE-data methods

- Hybrid residual losses

- Inverse problems and data assimilation

Why classical solvers remain essential

Checking a neural solution

- Formulation and error checks

Section overview

From data to PDE solvers

- Data-driven learning

- PDE-informed learning

Strong-form PINNs

Network design and boundary constraints

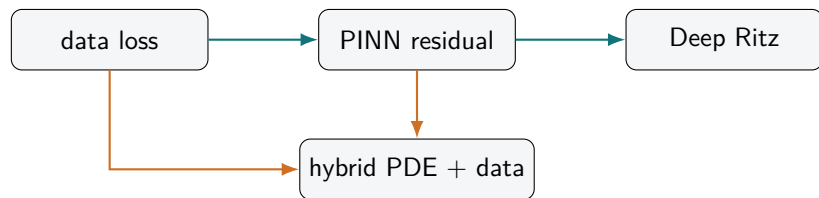
Variational and weak methods

Hybrid PDE-data methods

Why classical solvers remain essential

Checking a neural solution

Course map



$$\inf_{\theta \in \Theta} \mathcal{J}(\theta) \rightsquigarrow \text{function } u_{\theta} : \Omega \rightarrow \mathbb{R}^{d_u}.$$

The starting point: data-driven learning

- *What does a neural network learn before we use any PDE structure?*

$$\mathcal{D}_d = \{(x_i^d, y_i)\}_{i=1}^{N_d}, \quad x_i^d \in \mathbb{R}^d, \quad y_i \in \mathbb{R}^{d_u}.$$

$$\hat{\mathcal{J}}_d(\theta) = \frac{1}{N_d} \sum_{i=1}^{N_d} \ell(u_\theta(x_i^d), y_i), \quad \text{train } \theta \in \Theta \text{ to reduce } \hat{\mathcal{J}}_d.$$

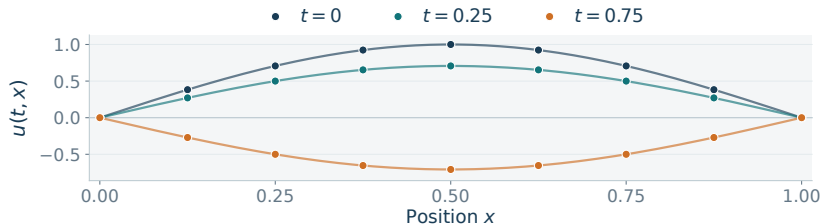
Interpretation

- We start by fitting a nonlinear trial function to observations.

d is the input dimension and d_u the output dimension; most examples have $d_u = 1$.

Supervised PDE data: solution samples

$$u_{tt} = u_{xx}, \quad u(t, x) = \sin(\pi x) \cos(\pi t), \quad x \in [0, 1].$$



$$\mathcal{D}_d = \{(t_i^d, x_i^d, y_i)\}_{i=1}^{N_d}, \quad \hat{\mathcal{J}}_d(\theta) = \frac{1}{N_d} \sum_{i=1}^{N_d} |u_\theta(t_i^d, x_i^d) - y_i|^2.$$

Synthetic noiseless samples $y_i = u(t_i^d, x_i^d)$, with fixed endpoints $u(t, 0) = u(t, 1) = 0$.
Training uses only the samples: the PDE is absent from this loss.

Use the PDE to train the network

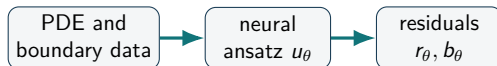
- *What changes when solution labels are unavailable but the governing equation is known?*

$$\mathcal{L}u = f \quad \text{in } \Omega, \quad \mathcal{B}u = g \quad \text{on } \partial\Omega$$

Solution labels y_i are unavailable, but

$\mathcal{L}u_\theta(x_i^r) - f(x_i^r)$ can be evaluated.

$\theta \mapsto u_\theta$ parametrises a nonlinear ansatz class.



Section overview

From data to PDE solvers

Strong-form PINNs

- Formulation and collocation

- Quadrature, stability, and error

- Training and approximation limits

Network design and boundary constraints

Variational and weak methods

Hybrid PDE-data methods

Why classical solvers remain essential

Checking a neural solution

A running elliptic model problem

- Which strong and weak notions of solution will we compare?

$$\begin{cases} -\nabla \cdot (A(x)\nabla u) = f & \text{in } \Omega, \\ u = g_D & \text{on } \Gamma_D, \\ A\nabla u \cdot n = g_N & \text{on } \Gamma_N. \end{cases}$$

$$A(x)\xi \cdot \xi \geq a_0|\xi|^2, \quad \Omega \subset \mathbb{R}^d \text{ bounded Lipschitz.}$$

Two solution concepts

- Strong: pointwise derivatives.
- Weak: variational derivatives.

Classical discretisation versus neural ansatz

$$V_h = \text{span}\{\phi_1, \dots, \phi_M\}, \quad u_h = \sum_{j=1}^M c_j \phi_j.$$

$$\mathcal{U}_\Theta = \{u_\theta : \theta \in \Theta\}, \quad \Theta \subset \mathbb{R}^P.$$

$$u_\theta(x) = W_L \sigma(\dots \sigma(W_1 x + b_1)).$$

method	unknowns	geometry
FEM/FDM	linear coefficients	mesh/stencil
NN PDE solver	nonlinear weights	sampled points

Strong residual minimisation

- *How can the PDE itself become a loss function?*

$$r_{\theta}(x) = \mathcal{L}[u_{\theta}](x) - f(x).$$

$$\mathcal{J}_r(\theta) = \|r_{\theta}\|_{L^2(\Omega)}^2 = \int_{\Omega} |\mathcal{L}[u_{\theta}](x) - f(x)|^2 dx.$$

$$\inf_{\theta \in \Theta} \mathcal{J}_r(\theta) \quad (\text{attainment is a separate question}).$$

From collocation points to the PINN objective

Choose interior points $X_r = \{x_i^r\}_{i=1}^{N_r}$ and boundary points $X_b = \{x_i^b\}_{i=1}^{N_b}$, with positive quadrature weights.

$$\hat{\mathcal{J}}_r(\theta) = \sum_{i=1}^{N_r} w_i^r |r_\theta(x_i^r)|^2,$$

$$\hat{\mathcal{J}}_b(\theta) = \sum_{i=1}^{N_b} w_i^b |b_\theta(x_i^b)|^2, \quad b_\theta = \mathcal{B}u_\theta - g.$$

Balance the interior and boundary equations

$$\hat{\mathcal{J}}_{\text{PINN}}(\theta) = \lambda_r \hat{\mathcal{J}}_r(\theta) + \lambda_b \hat{\mathcal{J}}_b(\theta).$$

The positive weights λ_r, λ_b set the relative scale and priority of the two terms. Training returns parameters $\hat{\theta}$.

The continuous boundary loss is $\mathcal{J}_b = \|b_\theta\|_{L^2(\partial\Omega)}^2$. Uniform Monte Carlo uses $w_i^r = |\Omega|/N_r$ and $w_i^b = |\partial\Omega|/N_b$.

Question 1: is the neural minimum attained?

- *Even for a well-posed PDE, must a best neural-network solution exist?*

Consider the uniquely solvable problem

$$-u'' = 0 \quad \text{in } (0, 1), \quad u(0) = 0, \quad u(1) = 1; \quad u(x) = x.$$

Fix any finite width $m \geq 1$ and use a shallow tanh network:

$$u_\theta(x) = c + \sum_{j=1}^m a_j \tanh(b_j x + d_j).$$

$$\mathcal{J}(v) = \int_0^1 |v''(x)|^2 dx + |v(0)|^2 + |v(1) - 1|^2.$$

Loss can approach zero without attaining it

We will construct networks with loss tending to zero, while no finite parameter vector attains zero loss.

A tanh sequence approaches the infimum

One hidden neuron already gives the sequence

$$v_n(x) = \frac{\tanh(x/n)}{\tanh(1/n)}, \quad n = 1, 2, \dots$$

$$v_n(0) = 0, \quad v_n(1) = 1, \quad v_n \longrightarrow x \quad \text{in } C^2[0, 1].$$

Indeed, Taylor expansion yields

$$v_n(x) = x + \frac{x - x^3}{3n^2} + O(n^{-4}), \quad v_n''(x) = -\frac{2x}{n^2} + O(n^{-4}).$$

$$\mathcal{J}(v_n) = \frac{4}{3n^4} + O(n^{-6}) \longrightarrow 0.$$

The output weight diverges while the functions converge

The output weight $a_1^{(n)} = 1/\tanh(1/n) \sim n$ diverges. The corresponding functions converge to the exact PDE solution.

Why no finite tanh network attains this minimum

- ▶ If $\mathcal{J}(u_\theta) = 0$, then $u_\theta'' = 0$ and the endpoint data force $u_\theta(x) = x$ throughout $(0, 1)$.
- ▶ A finite sum of tanh units is real analytic on $\mathbb{R}$. Equality on an interval would therefore imply $u_\theta(x) = x$ on all of $\mathbb{R}$.
- ▶ Every such finite tanh sum is bounded on $\mathbb{R}$; the function x is unbounded. This is a contradiction.

$$\inf_{\theta \in \Theta} \mathcal{J}(u_\theta) = 0, \quad \arg \min_{\theta \in \Theta} \mathcal{J}(u_\theta) = \emptyset.$$

Existence and approximation are different questions

The PDE has a unique solution. The neural infimum is approached arbitrarily well, but its limit lies outside this fixed finite tanh class.

The conclusion depends on the stated architecture: adding a linear skip connection makes $v(x) = x$ representable and removes this example.

Question 2: do small continuous residuals force convergence?

- If $\mathcal{J}(u_{\theta_n}) \rightarrow 0$ for the exact integral loss, must u_{θ_n} converge to the PDE solution in L^2 ?

Consider the semilinear Neumann problem

$$-u'' + \frac{u}{1+u^2} = 0 \quad \text{in } (0,1), \quad u'(0) = u'(1) = 0.$$

Multiplying by u and integrating gives

$$\int_0^1 \left(|u'|^2 + \frac{u^2}{1+u^2} \right) dx = 0, \quad \text{so the unique solution is } u = 0.$$

The continuous residual loss

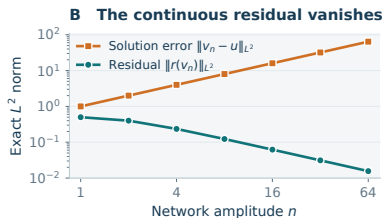
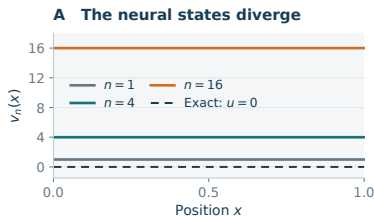
$$\mathcal{J}(v) = \int_0^1 \left| -v'' + \frac{v}{1+v^2} \right|^2 dx + |v'(0)|^2 + |v'(1)|^2.$$

All integrals are exact. This example tests global residual-to-state stability, without introducing sampling or quadrature.

Counterexample: vanishing continuous loss, diverging state

One tanh neuron represents a constant sequence:

$$v_n(x) = \frac{n}{\tanh(1)} \tanh(0x + 1) = n, \quad v'_n(0) = v'_n(1) = 0.$$



$$\mathcal{J}(v_n) = \frac{n^2}{(1 + n^2)^2} \rightarrow 0, \quad \|v_n - u\|_{L^2(0,1)} = n \rightarrow \infty.$$

Uniqueness alone gives no global residual-to-error bound: $s/(1 + s^2) \rightarrow 0$ as $s \rightarrow \infty$.

Example: Poisson PINN

$$\begin{cases} -\Delta u = f & \text{in } \Omega, \\ u = g & \text{on } \partial\Omega. \end{cases}$$

$$r_\theta(x) = -\sum_{j=1}^d \partial_{x_j x_j} u_\theta(x) - f(x).$$

$$\begin{aligned} \hat{\mathcal{J}}_{\text{PINN}}(\theta) &= \lambda_r \frac{|\Omega|}{N_r} \sum_{i=1}^{N_r} |r_\theta(x_i^r)|^2 \\ &\quad + \lambda_b \frac{|\partial\Omega|}{N_b} \sum_{i=1}^{N_b} |u_\theta(x_i^b) - g(x_i^b)|^2. \end{aligned}$$

Example: heat equation PINN

$$\begin{cases} \partial_t u - \kappa \Delta u = f & \text{in } Q = (0, T) \times \Omega, \\ u(0, x) = u_0(x) & x \in \Omega, \\ \mathcal{B}u = g & \text{on } \Sigma = (0, T) \times \partial\Omega. \end{cases}$$

$$r_\theta(t, x) = \partial_t u_\theta(t, x) - \kappa \Delta_x u_\theta(t, x) - f(t, x).$$

$$\begin{aligned} \mathcal{J}(\theta) = & \lambda_r \|r_\theta\|_{L^2(Q)}^2 + \lambda_0 \|u_\theta(0, \cdot) - u_0\|_{L^2(\Omega)}^2 \\ & + \lambda_b \|\mathcal{B}u_\theta - g\|_{L^2(\Sigma)}^2. \end{aligned}$$

$\Sigma = (0, T) \times \partial\Omega$ is the lateral boundary; u_0 is the initial datum.

Approximating the residual integral

$$\mathcal{J}_r(\theta) = \int_{\Omega} |r_{\theta}(x)|^2 dx.$$

$$\hat{\mathcal{J}}_r(\theta) = \sum_{i=1}^{N_r} w_i^r |r_{\theta}(x_i^r)|^2.$$

$\underbrace{\mathcal{J}_r(\hat{\theta})}_{\text{continuum residual}} \quad \text{need not equal} \quad \underbrace{\hat{\mathcal{J}}_r(\hat{\theta})}_{\text{training residual}}$

Relating the two requires control of the quadrature error.

Two ways to approximate the residual integral

For $\varphi_\theta = |r_\theta|^2$, write $I(\varphi) = \int_\Omega \varphi \, dx$.

Deterministic quadrature: regularity controls the error

$$Q_N(\varphi) = \sum_{i=1}^N w_i \varphi(x_i), \quad |I(\varphi) - Q_N(\varphi)| \leq Ch^q \|\varphi\|_{W^{q,1}(\Omega)}.$$

Resolving the network values alone is insufficient: derivatives of the squared residual enter the quadrature estimate.

Monte Carlo: variance controls the error

$$x_i \stackrel{\text{iid}}{\sim} \text{Unif}(\Omega), \quad \widehat{I}_N(\varphi) = \frac{|\Omega|}{N} \sum_{i=1}^N \varphi(x_i).$$

$$\mathbb{E}[\widehat{I}_N] = I(\varphi), \quad \text{Var}[\widehat{I}_N] = \frac{|\Omega|^2}{N} \text{Var}[\varphi(X)].$$

The root-mean-square rate is $N^{-1/2}$; its constant depends on the variance.

Residual norm versus solution error

- *When does a small residual actually imply a small solution error?*

$$e_\theta = u_\theta - u.$$

$$\mathcal{L}e_\theta = r_\theta, \quad \mathcal{B}e_\theta = b_\theta.$$

These residual identities assume linear operators $\mathcal{L}$ and $\mathcal{B}$.

$$\|e_\theta\|_X \leq C_{\text{stab}}(\|r_\theta\|_Y + \|b_\theta\|_Z)$$

Key point

- A stability estimate tells us when the residual controls solution error.

When a strong residual requires too much regularity

$\mathcal{L} : H^s(\Omega) \rightarrow L^2(\Omega)$ requires $u \in H^s(\Omega)$.

$-\Delta u = f$ in $L^2 \Rightarrow u \in H^2(\Omega)$ only under regularity assumptions.

$f \in L^2(\Omega)$ still gives $\Delta u = -f \in L^2(\Omega)$,

but on a nonconvex domain, $u \in H^2(\Omega)$ may fail.

Why the loss can be nonconvex

Even for a linear PDE, the network depends nonlinearly on its parameters. The loss depends on the parameters through:

$$\theta \mapsto u_\theta \mapsto \mathcal{L}u_\theta \mapsto \|\mathcal{L}u_\theta - f\|_{L^2(\Omega)}^2.$$

The loss need not satisfy the convexity inequality:

$$\mathcal{J}(\alpha\theta_1 + (1 - \alpha)\theta_2) \not\leq \alpha\mathcal{J}(\theta_1) + (1 - \alpha)\mathcal{J}(\theta_2).$$

The inequality can fail for some θ_1, θ_2 and $0 < \alpha < 1$; interpolating weights does not interpolate network outputs.

What this means for training

The final loss depends on initialization and the optimizer. Check the PDE error even when the parameter gradient is small.

$$\theta_K = \text{Optimizer}_K(\theta_0, \hat{\mathcal{J}}, \eta, \text{batching}).$$

K is the number of updates and η the learning rate; training uses the sampled objective $\hat{\mathcal{J}}$.

Optimization error

$$\begin{aligned}\mathcal{J}(\theta_K) - \inf_{\theta \in \Theta} \mathcal{J}(\theta) &= \underbrace{\mathcal{J}(\theta_K) - \hat{\mathcal{J}}(\theta_K)}_{\text{quadrature/generalization}} \\ &\quad + \underbrace{\hat{\mathcal{J}}(\theta_K) - \inf_{\theta \in \Theta} \hat{\mathcal{J}}(\theta)}_{\text{optimization}} \\ &\quad + \underbrace{\inf_{\theta \in \Theta} \hat{\mathcal{J}}(\theta) - \inf_{\theta \in \Theta} \mathcal{J}(\theta)}_{\text{quadrature bias}}.\end{aligned}$$

Training loss and the full PDE error budget

$$\hat{\mathcal{J}}(\theta_K) \downarrow \not\Rightarrow \|u_{\theta_K} - u\|_X \downarrow.$$

Sources of PDE error

PDE error $\lesssim$ approximation + quadrature
+ optimization + penalty + modeling.

- ▶ A stability estimate must turn the chosen residual into a state-error bound.
- ▶ Quadrature and boundary control must remain reliable for the trained network.
- ▶ Successful optimization reduces only one part of the error budget; architecture, sampling, and model accuracy affect the other parts.

Python lab 1: solve Poisson without solution labels

$$-u'' = \pi^2 \sin(\pi x), \quad x \in (0, 1), \quad u(0) = u(1) = 0.$$

```
# Imports: numpy as np, torch, and torch.nn as nn
initial = nn.Sequential(
    nn.Linear(1, 32), nn.Tanh(),
    nn.Linear(32, 32), nn.Tanh(),
    nn.Linear(32, 1)
)
nodes, quadrature_weights = np.polynomial.legendre.leggauss(
    QUADRATURE_NODES)
x = torch.tensor(((nodes + 1.0) / 2.0)[: , None], requires_grad=True)
weights = torch.tensor((quadrature_weights / 2.0)[: , None])

hard_model = Solution(initial, hard_boundary=True)
```

Network and training setup

Use $u_\theta = x(1-x)N_\theta$, with raw network N_θ ; seed 7, float64 on CPU. Use the exact solution $u = \sin(\pi x)$ to evaluate the error after training.

Python lab 1: differentiate, form the loss, train

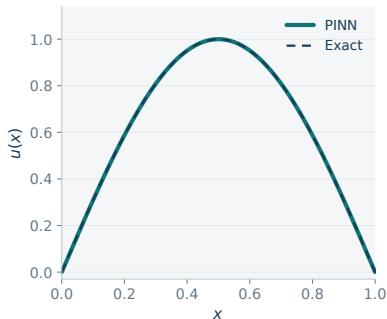
```
def derivative(y, x):  
    return torch.autograd.grad(  
        y, x, torch.ones_like(y), create_graph=True)[0]  
  
def pinn_loss(model, x, weights):  
    u = model(x)  
    u_x = derivative(u, x)  
    u_xx = derivative(u_x, x)  
    residual = (-u_xx - forcing(x)) / math.pi**2  
    return (weights * residual.square()).sum()  
optimizer = torch.optim.Adam(hard_model.parameters(), lr=1e-3)  
optimizer.zero_grad()  
loss = pinn_loss(hard_model, x, weights)  
loss.backward()  
optimizer.step()
```

Differentiate network values (u) in x for the residual, and in θ to train.

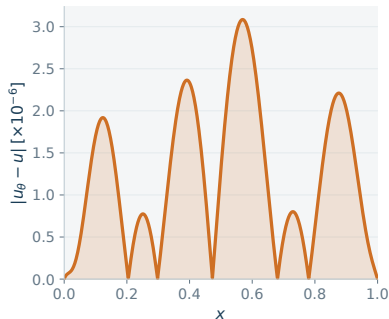
128 Gauss–Legendre nodes; 2,000 Adam updates + 300 L-BFGS iterations. Residual divided by π^2 ; boundaries enforced by $x(1 - x)$.

Python output: inspect the solution and its error

A A learned solution



B Error at evaluation points



Measured on 1,001 evaluation points

Relative L^2 error: 2.11×10^{-6} ; maximum absolute error: 3.08×10^{-6} .
The endpoints are exactly zero by construction.

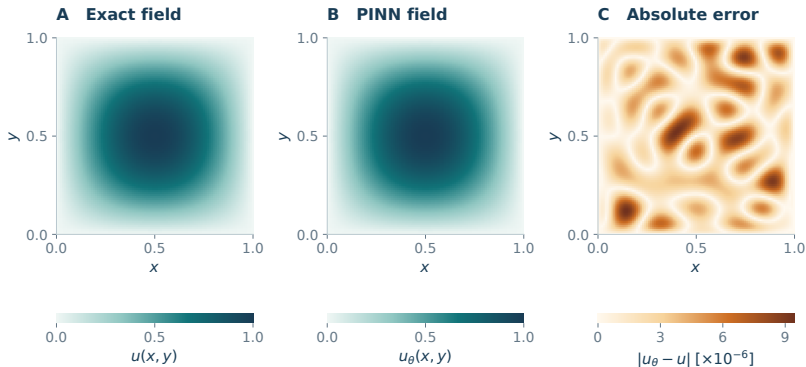
The error panel resolves differences hidden by overlapping solution curves. All reported errors are numerical grid estimates.

Python lab 2: the same idea in two dimensions

$$-\Delta u = 2\pi^2 \sin(\pi x) \sin(\pi y), \quad u|_{\partial(0,1)^2} = 0.$$

```
def normalized_residual(model, xy):  
    """Return  $(-\Delta u_{\text{theta}} - f)/(2\pi^2)$ ."""  
    u = model(xy)  
    grad_u = torch.autograd.grad(  
        u.sum(), xy, create_graph=True)[0]  
    u_xx = torch.autograd.grad(  
        grad_u[:, 0].sum(), xy, create_graph=True)[0][:, 0:1]  
    u_yy = torch.autograd.grad(  
        grad_u[:, 1].sum(), xy, create_graph=True)[0][:, 1:2]  
    x, y = xy[:, 0:1], xy[:, 1:2]  
    f = 2 * math.pi**2 * torch.sin(math.pi*x) * torch.sin(math.pi*y)  
    return (-u_xx - u_yy - f) / (2 * math.pi**2)
```

Python output: a field, a reference, an error map



Validate on a separate 101×101 mesh

Relative L^2 error: 6.43×10^{-6} ; maximum absolute error: 9.45×10^{-6} .

Exact and learned fields share one color scale. The error uses its own scale to reveal where the approximation is least accurate.

Section overview

From data to PDE solvers

Strong-form PINNs

Network design and boundary constraints

- Regularity and architecture

- Boundary conditions

Variational and weak methods

Hybrid PDE-data methods

Why classical solvers remain essential

Checking a neural solution

Activation functions: derivative order matters

- *How smooth must the network be for the chosen differential operator?*

$\mathcal{L}$ contains derivatives up to order s .

A pointwise residual needs derivatives through order s
at the collocation points.

$\sigma \in C^s \Rightarrow u_\theta \in C^s(\Omega)$ is sufficient for these derivatives to exist.

ReLU: classical derivatives and distributional derivatives

$$\sigma(z) = \max\{0, z\}, \quad \sigma'(z) = \mathbf{1}_{z>0}, \quad \sigma''(z) = 0 \quad \text{a.e.}$$

A ReLU network is piecewise affine. Inside each linear region, $D^2 u_\theta = 0$, so its classical Laplacian cannot match a smooth nonzero forcing.

The missing second derivative lives at the kinks

$$D^2 \text{ReLU} = \delta_0 \quad \text{in distributions.}$$

Automatic differentiation usually returns zero away from the kinks; it does not return this singular measure.

Thus a strong second-order residual can miss the network's singular second derivatives, even when its values look accurate.

When ReLU can still be sensible

First-order PDE: $\mathcal{L}u = b \cdot \nabla u + cu$.

Weak formulations: derivatives are shifted to tests.

Deep Ritz for Poisson: $\mathcal{E}(v) = \frac{1}{2} \int_{\Omega} |\nabla v|^2 dx - \int_{\Omega} fv dx$.

$\nabla u_{\theta} \in L^{\infty}$ piecewise constant is admissible in H^1 .

Choose the activation for the formulation

$\tanh z$	: C^∞ , saturating,
$\text{softplus}_\beta(z) = \beta^{-1} \log(1 + e^{\beta z})$	: C^∞ ,
$\sin(\omega z)$	: C^∞ , oscillatory,
$z\sigma_{\text{sigmoid}}(z)$	: SiLU/Swish, smooth.

Match the derivatives used by the loss

For a strong residual of order s , choose $\sigma \in C^s$ with numerically stable derivatives through order s .

Poisson energy needs only first derivatives: $\sigma \in W_{\text{loc}}^{1,\infty}(\mathbb{R})$ suffices for $u_\theta \in H^1(\Omega)$ on a bounded domain.

Smoothness makes the derivatives available; it does not guarantee good conditioning.

Boundary constraints and the Dirichlet penalty

The constrained problem is

$$\inf_{v \in X} \mathcal{J}_r(v) \quad \text{subject to} \quad \mathcal{B}v = g.$$

A penalty replaces exact feasibility by a weighted boundary mismatch.
For $u = g_D$ on $\Gamma_D \subset \partial\Omega$,

$$\mathcal{J}_D(\theta) = \int_{\Gamma_D} |u_\theta - g_D|^2 \, ds,$$

$$\hat{\mathcal{J}}_D(\theta) = \sum_{i=1}^{N_D} w_i^D |u_\theta(x_i^D) - g_D(x_i^D)|^2 \approx \mathcal{J}_D(\theta).$$

Soft enforcement

$$\mathcal{J}(\theta) = \mathcal{J}_r(\theta) + \lambda_D \mathcal{J}_D(\theta).$$

Finite λ_D trades interior fit against boundary accuracy. The exact constraint is recovered only in an ideal penalty limit.

The subscript D identifies the Dirichlet boundary; $x_i^D \in \Gamma_D$.

Neumann and Robin penalties

$$A\nabla u \cdot n = g_N \quad \Rightarrow \quad \mathcal{J}_N(\theta) = \int_{\Gamma_N} |A\nabla u_\theta \cdot n - g_N|^2 \, ds.$$

$$\alpha u + A\nabla u \cdot n = g_R \quad \Rightarrow \quad \mathcal{J}_R(\theta) = \int_{\Gamma_R} |\alpha u_\theta + A\nabla u_\theta \cdot n - g_R|^2 \, ds.$$

Initial conditions are boundary conditions in space-time

$$Q = (0, T) \times \Omega, \quad \Sigma_0 = \{0\} \times \Omega, \quad \Sigma = (0, T) \times \partial\Omega.$$

$$\mathcal{J}_0(\theta) = \int_{\Omega} |u_{\theta}(0, x) - u_0(x)|^2 dx.$$

$$\mathcal{J}(\theta) = \lambda_r \mathcal{J}_r(\theta) + \lambda_0 \mathcal{J}_0(\theta) + \lambda_b \mathcal{J}_b(\theta).$$

Penalty parameter asymptotics

$$\theta_{\lambda_b} \in \arg \min_{\theta \in \Theta} (\mathcal{J}_r(\theta) + \lambda_b \mathcal{J}_b(\theta)).$$

$\lambda_b \uparrow \infty :$ $\mathcal{J}_b(\theta_{\lambda_b}) \downarrow$ but conditioning deteriorates.

$$\nabla_{\theta} \mathcal{J} = \nabla_{\theta} \mathcal{J}_r + \lambda_b \nabla_{\theta} \mathcal{J}_b.$$

This ideal comparison assumes exact minimisers exist for each λ_b .

Encode Dirichlet data directly in the network

Let N_θ be an unconstrained network. Choose a lifting G and a boundary-vanishing factor ρ such that

$$G|_{\partial\Omega} = g, \quad \rho|_{\partial\Omega} = 0, \quad \rho > 0 \text{ in } \Omega.$$

$$\boxed{u_\theta(x) = G(x) + \rho(x)N_\theta(x)} \implies u_\theta|_{\partial\Omega} = g.$$

Homogeneous data are the special case $G = 0$

Then $u_\theta = \rho N_\theta$ satisfies zero Dirichlet data for every θ . The training objective only needs the interior residual:

$$\mathcal{J}_r(\theta) = \int_{\Omega} |\mathcal{L}[G + \rho N_\theta] - f|^2 dx.$$

Boundary enforcement is exact when G and ρ satisfy their trace conditions exactly.

Hard constraints on boxes and in time

Zero Dirichlet data on the unit box

$$\Omega = (0, 1)^d, \quad \rho(x) = \prod_{j=1}^d x_j(1 - x_j),$$
$$u_\theta(x) = \rho(x)N_\theta(x) \implies u_\theta|_{\partial\Omega} = 0.$$

Initial data and compatible spatial boundary data

For $u(0, x) = u_0(x)$, use $u_\theta(t, x) = u_0(x) + tN_\theta(t, x)$. To encode both initial and boundary conditions, use

$$u_\theta(t, x) = G(t, x) + t\rho(x)N_\theta(t, x).$$

Choose $G(0, x) = u_0(x)$ and $G = g$ on the spatial boundary.

The initial and boundary data must be compatible where their boundaries meet.

Boundary penalties versus hard constraints

approach	advantage	risk
penalty	easy, general	tuning, constraint drift
hard encoding	exact Dirichlet	geometry, regularity, expressivity
Nitsche/weak	variationally consistent	more derivation, parameters

Choose based on geometry, regularity, and boundary accuracy.

Python lab 3: sweep the boundary penalty

Keep the 1D Poisson problem; replace $x(1-x)N_\theta(x)$ by $N_\theta(x)$.

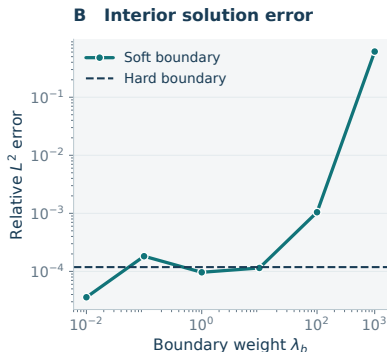
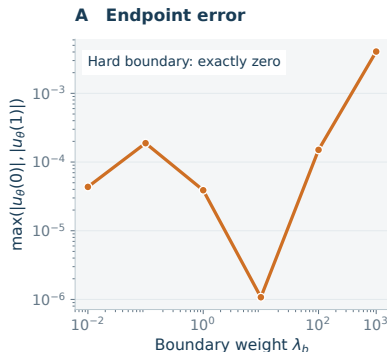
```
def soft_loss(model, x, weights, penalty):  
    boundary = torch.tensor([[0.0], [1.0]])  
    boundary_loss = model(boundary).square().mean()  
    return pinn_loss(model, x, weights) + penalty * boundary_loss
```

Change one setting at a time

$\lambda_b \in \{0.01, 0.1, 1, 10, 100, 1000\}$; identical initial weights, collocation points, learning rate, and 2000 Adam updates. The hard-constraint run gets the same update budget.

The residual is divided by π^2 , so the weight sweep refers to that specific scaling. No L-BFGS refinement is used in this comparison.

Python output: boundary accuracy is only one diagnostic



Read both panels together

Check the interior error as well as the boundary error. The hard-constraint baseline has zero endpoint error exactly.

These results use one seed and one training budget. Repeat the sweep with other settings before choosing a penalty weight.

Section overview

From data to PDE solvers

Strong-form PINNs

Network design and boundary constraints

Variational and weak methods

- Deep Ritz methods

- Weak formulations

Hybrid PDE-data methods

Why classical solvers remain essential

Checking a neural solution

From a weak formulation to an energy

For homogeneous Dirichlet data, take $V = H_{\Gamma_D}^1(\Omega)$ and seek

$$u \in V : \quad a(u, v) = F(v) \quad \forall v \in V,$$

$$a(u, v) = \int_{\Omega} A \nabla u \cdot \nabla v + cuv \, dx,$$

$$F(v) = \int_{\Omega} fv \, dx + \int_{\Gamma_N} g_N v \, ds.$$

If a is symmetric, continuous and coercive, define

$$\mathcal{E}(v) = \frac{1}{2}a(v, v) - F(v), \quad \delta\mathcal{E}(u)[v] = a(u, v) - F(v).$$

$$\boxed{u \in \arg \min_{v \in V} \mathcal{E}(v)} \quad \Longleftrightarrow \quad \text{the weak PDE.}$$

Nonzero Dirichlet data require a lifting and minimisation over the corresponding affine space.

Poisson energy

$$-\Delta u = f, \quad u|_{\partial\Omega} = 0.$$

$$V = H_0^1(\Omega), \quad \mathcal{E}(v) = \frac{1}{2} \int_{\Omega} |\nabla v|^2 \, dx - \int_{\Omega} f v \, dx.$$

$$\delta\mathcal{E}(u)[v] = \int_{\Omega} \nabla u \cdot \nabla v \, dx - \int_{\Omega} f v \, dx.$$

Deep Ritz: train with first derivatives

Choose an admissible neural trial class and approximate the Poisson energy:

$$\mathcal{U}_\Theta \subset V, \quad \hat{\theta} \in \Theta : \quad \text{trained to reduce } \hat{\mathcal{E}}(u_\theta).$$

$$\hat{\mathcal{E}}(u_\theta) = \frac{1}{2} \sum_{i=1}^N w_i |\nabla u_\theta(x_i)|^2 - \sum_{i=1}^N w_i f(x_i) u_\theta(x_i).$$

What changes from a strong PINN?

- ▶ The Poisson residual requires Δu_θ ; the energy only requires ∇u_θ .
- ▶ First derivatives lower the regularity required of the trial functions.
- ▶ No labelled solution data are used: f , the domain and boundary conditions define the training objective.

Energy excess controls solution error

For the exact weak solution u and any admissible v , stationarity cancels the linear error term:

$$\mathcal{E}(v) - \mathcal{E}(u) = \frac{1}{2} a(v - u, v - u) = \frac{1}{2} \|v - u\|_a^2.$$

$$\|e\|_a^2 := a(e, e), \quad \alpha \|e\|_V^2 \leq \|e\|_a^2 \leq M \|e\|_V^2.$$

$$\|v - u\|_V \leq \alpha^{-1/2} \|v - u\|_a = \sqrt{\frac{2}{\alpha} (\mathcal{E}(v) - \mathcal{E}(u))}$$

The energy gap is exactly a squared energy-norm error. Coercivity transfers this control to the desired Sobolev norm.

Coercivity from ellipticity

$$A(x)\xi \cdot \xi \geq a_0|\xi|^2, \quad c(x) \geq 0.$$

$$a(v, v) = \int_{\Omega} A \nabla v \cdot \nabla v + cv^2 \, dx \geq a_0 \|\nabla v\|_{L^2}^2.$$

$$V = H_0^1(\Omega) : \quad \|v\|_{L^2} \leq C_P \|\nabla v\|_{L^2}.$$

Thus $a(v, v) \geq \frac{a_0}{1+C_P^2} \|v\|_{H^1}^2$: ellipticity controls gradients, and the Dirichlet condition controls constants.

Best approximation in energy norm

$$u_{\Theta}^* \in \arg \min_{v \in \mathcal{U}_{\Theta}} \mathcal{E}(v).$$

$$\mathcal{E}(u_{\Theta}^*) - \mathcal{E}(u) = \frac{1}{2} \|u_{\Theta}^* - u\|_a^2.$$

Deep Ritz inherits the approximation properties of $\mathcal{U}_{\Theta}$ in H^1 .

Here u_{Θ}^* denotes a best neural approximation, assuming the infimum is attained.

Boundary penalty in Deep Ritz

$$\mathcal{E}_{\lambda_b}(v) = \mathcal{E}(v) + \frac{\lambda_b}{2} \int_{\partial\Omega} |v - g|^2 \, ds.$$

$$\begin{aligned} \delta\mathcal{E}_{\lambda_b}(v)[w] &= a(v, w) - F(w) \\ &\quad + \lambda_b \int_{\partial\Omega} (v - g)w \, ds. \end{aligned}$$

$\lambda_b < \infty \quad \Rightarrow \quad$ Dirichlet data imposed weakly, not exactly.

Natural boundary conditions in energy form

$$\begin{aligned}\mathcal{E}(v) = & \frac{1}{2} \int_{\Omega} A \nabla v \cdot \nabla v \, dx \\ & - \int_{\Omega} f v \, dx - \int_{\Gamma_N} g_N v \, ds.\end{aligned}$$

$$\delta \mathcal{E}(u)[w] = 0 \quad \Rightarrow \quad A \nabla u \cdot n = g_N \text{ on } \Gamma_N.$$

Neumann data are natural; Dirichlet data are essential.

Deep Ritz for nonlinear energies

$$\mathcal{E}(v) = \int_{\Omega} W(\nabla v, v, x) \, dx - \int_{\Omega} f v \, dx.$$

$$\delta \mathcal{E}(u)[w] = 0 \quad \Rightarrow \quad -\nabla \cdot W_p(\nabla u, u, x) + W_z(\nabla u, u, x) = f.$$

Nonconvex W can create multiple stationary points.

W_p and W_z are derivatives with respect to the gradient and state arguments of $W(p, z, x)$.

Two ways coercivity can weaken in practice

Sampling. A continuum energy can control the state error,

$$\mathcal{E}(v) - \mathcal{E}(u) \geq c \|v - u\|_V^2,$$

while a sampled objective has no corresponding guarantee without further control:

$$\widehat{\mathcal{E}}(v) - \widehat{\mathcal{E}}(u_{\widehat{\theta}}) \not\geq c \left\| v - u_{\widehat{\theta}} \right\|_V^2 \quad \text{in general.}$$

Boundary penalty. Gradients and traces control H^1 :

$$\mathcal{E}_{\lambda_b}(v) = \frac{1}{2} \|\nabla v\|_{L^2}^2 - \int_{\Omega} f v \, dx + \frac{\lambda_b}{2} \|v - g\|_{L^2(\partial\Omega)}^2.$$

A small λ_b gives weak control of constants and boundary modes.

In the sampled comparison, $u_{\widehat{\theta}}$ denotes a minimiser, if one exists.

Python lab 4: replace the strong residual by energy

For the same Poisson problem and hard Dirichlet ansatz,

$$\mathcal{E}(u_\theta) = \int_0^1 \left[\frac{1}{2} (u'_\theta)^2 - \pi^2 \sin(\pi x) u_\theta \right] dx.$$

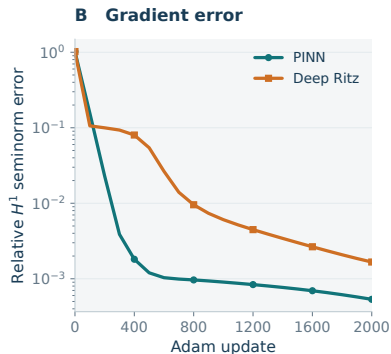
```
def ritz_loss(model, x, weights):  
    u = model(x)  
    u_x = derivative(u, x)  
    return (weights * (0.5 * u_x.square() - forcing(x) * u)).sum()
```

First derivatives suffice

Use Gauss–Legendre nodes and weights on $[0, 1]$. The same network initialization isolates the change of objective.

128 quadrature nodes; 2,000 Adam updates + 300 L-BFGS iterations.

Python output: compare errors, not raw loss values



Final evaluation after optimizer refinement

	relative L^2 error	relative H^1 seminorm error
PINN	2.11×10^{-6}	1.14×10^{-5}
Deep Ritz	3.85×10^{-6}	3.33×10^{-5}

Curves show Adam only; the table includes subsequent L-BFGS refinement. Equal updates do not imply equal runtime. One seed, one smooth problem.

Weak residuals and finite test spaces

A weak formulation transfers derivatives to test functions and can accommodate solutions without classical second derivatives:

$$\mathcal{R}_\theta(v) = a(u_\theta, v) - F(v), \quad v \in V.$$

The exact solution satisfies $a(u, v) = F(v)$ for every $v \in V$. For pure diffusion and $v_j \in H_0^1(\Omega)$, the neural residual moments are

$$\mathcal{R}_\theta(v_j) = \int_{\Omega} A \nabla u_\theta \cdot \nabla v_j \, dx - \int_{\Omega} f v_j \, dx.$$

Choose a finite collection of tests and minimise

$$\mathcal{J}_{\text{weak}}(\theta) = \frac{1}{M} \sum_{j=1}^M |\mathcal{R}_\theta(v_j)|^2.$$

This tests selected directions, rather than every direction in V .

The moments above are exact integrals. Numerical quadrature produces $\hat{\mathcal{J}}_{\text{weak}}$.

Variational PINNs: neural trials and polynomial tests

Variational PINNs use spatially extended, admissible tests, often polynomials. The trial family and test space play different roles:

$$\mathcal{U}_\Theta = \{u_\theta : \theta \in \Theta\}, \quad V_M = \text{span}\{v_1, \dots, v_M\} \subset V.$$

$$\mathcal{J}_{\text{VPINN}}(\theta) = \frac{1}{M} \sum_{j=1}^M |\mathcal{R}_\theta(v_j)|^2.$$

This has a Petrov–Galerkin structure: a nonlinear neural trial family and a fixed test space. The choice of tests determines which residual features are detected.

Local variational residuals and hp-refinement

Partition Ω into cells $K \in \mathcal{T}_h$, while keeping a global neural trial function. For local tests vanishing on ∂K ,

$$\mathcal{R}_{\theta,K}(v_{K,j}) = \int_K A \nabla u_\theta \cdot \nabla v_{K,j} \, dx - \int_K f v_{K,j} \, dx,$$

$$\mathcal{J}_{\text{local}}(\theta) = \sum_{K \in \mathcal{T}_h} \sum_j |\mathcal{R}_{\theta,K}(v_{K,j})|^2.$$

Two ways to enrich the tests give the hp-VPINN idea:

h-refinement: $h_K = \text{diam}(K) \downarrow$,

p-refinement: $\deg v_{K,j} \uparrow$.

Smaller cells localise residual information; richer polynomials probe more variation inside each cell. The mesh organises the tests; the neural trial function remains global.

If a local test does not vanish on ∂K , include the boundary-flux term.

Mixed formulations and local conservation

Introduce the flux as a second unknown to replace a second-order equation by a first-order system:

$$\begin{cases} q + A \nabla u = 0, \\ \nabla \cdot q = f. \end{cases}$$

Train (q_θ, u_θ) using these residuals and the boundary conditions. The divergence equation also yields a balance on each cell:

$$\int_{\partial K} q \cdot n \, ds = \int_K f \, dx.$$

An additional conservation penalty can target these balances directly:

$$\mathcal{J}_{\text{cons}}(\theta) = \sum_{K \in \mathcal{T}_h} \left| \int_{\partial K} q_\theta \cdot n \, ds - \int_K f \, dx \right|^2.$$

Small pointwise training residuals do not automatically enforce exact local conservation. Flux accuracy and cell balances are useful separate diagnostics.

Discontinuous or low-regularity solutions

Conservation law: $\partial_t u + \nabla \cdot \mathbf{f}_{\text{flux}}(u) = 0$.

$$\int_Q [u \partial_t \phi + \mathbf{f}_{\text{flux}}(u) \cdot \nabla \phi] \, dx \, dt + \int_{\Omega} u_0 \phi(0, \cdot) \, dx = 0.$$

Weak/entropy formulations are essential.

$Q = (0, T) \times \Omega$; take smooth tests ϕ vanishing near $t = T$ and the spatial boundary.

Comparing residual and energy formulations

The choice of objective determines which information is measured:

$$\begin{aligned} \text{strong: } & \|\mathcal{L}u_\theta - f\|_{L^2}^2, & \text{energy: } & \mathcal{E}(u_\theta), \\ \text{finite tests: } & \frac{1}{M} \sum_j |\mathcal{R}_\theta(v_j)|^2. \end{aligned}$$

method	derivatives	objective	main issue
strong PINN	high	strong residual	regularity
Deep Ritz	low	energy	energy form
VPINN	low/medium	test moments	test design

For the same PDE, these objectives may share the exact solution but behave differently after restricting the trial family, discretising integrals, and optimising. Error control requires the stability assumptions appropriate to the chosen formulation.

Section overview

From data to PDE solvers

Strong-form PINNs

Network design and boundary constraints

Variational and weak methods

Hybrid PDE-data methods

- Hybrid residual losses

- Inverse problems and data assimilation

Why classical solvers remain essential

Checking a neural solution

Hybrid losses: combining PDE constraints and data

Observations add information where the PDE or boundary constraints alone may be insufficient. For $\mathcal{D}_d = \{(x_i^d, y_i)\}_{i=1}^{N_d}$ with $y_i \approx u(x_i^d)$,

$$\hat{\mathcal{J}}_d(\theta) = \frac{1}{N_d} \sum_{i=1}^{N_d} |u_\theta(x_i^d) - y_i|^2.$$

The hybrid training objective is

$$\boxed{\hat{\mathcal{J}}(\theta) = \lambda_r \hat{\mathcal{J}}_r(\theta) + \lambda_b \hat{\mathcal{J}}_b(\theta) + \lambda_d \hat{\mathcal{J}}_d(\theta).}$$

This combines three objectives with chosen weights:

$$\min_{\theta} (\hat{\mathcal{J}}_r(\theta), \hat{\mathcal{J}}_b(\theta), \hat{\mathcal{J}}_d(\theta)).$$

The weights select a trade-off between PDE agreement, boundary agreement, and data fit. With noisy data or an imperfect PDE model, all three losses may not be simultaneously small; scales and uncertainty should inform the weights.

What data resolve, and when they conflict

Resolving a nullspace. With compatible Neumann data,

$$\begin{cases} -\Delta u = f & \text{in } \Omega, \\ \partial_n u = g_N & \text{on } \partial\Omega, \end{cases} \quad u + c \text{ is also a solution.}$$

PDE and boundary residuals cannot determine c . Observations of u can fix this additive constant.

Handling noise. Measurements generally satisfy

$$y_i = u(x_i^d) + \varepsilon_i, \quad \hat{\mathcal{J}}_d(\theta) = \frac{1}{N_d} \sum_i |u_\theta(x_i^d) - y_i|^2.$$

A very large data weight may fit noise; a very large PDE weight may suppress useful evidence of model error. Relative to the other weights,

$\lambda_d \uparrow$: prioritise data fit, $\lambda_r \uparrow$: prioritise PDE agreement.

Thus data can remove nonuniqueness while also introducing a competing objective.

Inverse problems and identifiability

An inverse problem learns both the solution and an unknown PDE parameter:

$$\mathcal{L}_\mu u = f, \quad \mu \in \mathbb{R}^q, \quad r_{\theta, \mu} = \mathcal{L}_\mu u_\theta - f.$$

Train θ and μ jointly with

$$\hat{\mathcal{J}}_{\text{inv}}(\theta, \mu) = \lambda_r \hat{\mathcal{J}}_r(\theta, \mu) + \lambda_b \hat{\mathcal{J}}_b(\theta) + \lambda_d \hat{\mathcal{J}}_d(\theta).$$

Data can identify μ only if different admissible parameters produce observably different states. If

$$\begin{aligned} \mathcal{L}_{\mu_1} u_1 &= \mathcal{L}_{\mu_2} u_2 = f, & \mathcal{B}u_1 &= \mathcal{B}u_2 = g, \\ u_1(x_i^d) &= u_2(x_i^d) \quad \text{for every observation,} \end{aligned}$$

then the two pairs are indistinguishable from these constraints. Uniqueness requires injectivity of the observation map on admissible pairs; robust recovery also requires stability with respect to measurement errors.

Python lab 5: infer a coefficient from sparse data

$$-\kappa u'' = \pi^2 \sin(\pi x), \quad u(0) = u(1) = 0, \quad \text{true } \kappa = 0.7.$$

```
# Inside the model; raw_k is a trainable nn.Parameter.
@property
def k(self):
    return F.softplus(self.raw_k) + 1e-6

# r = (-k * u_xx - f) / pi**2; data_weight = 5.0
r = normalized_residual(model, x_r)
loss_physics = r.square().mean()
loss_data = (model(x_obs) - y_obs).square().mean()
loss = loss_physics + data_weight * loss_data
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
```

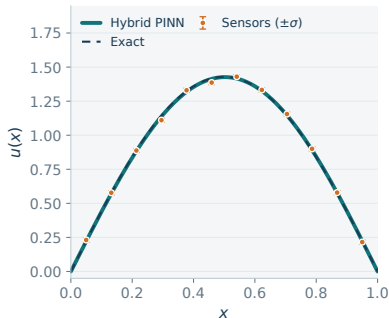
A synthetic inverse problem

12 sensors with independent Gaussian noise, $\sigma = 0.0143$ (1% of peak amplitude), seed 23. The network and κ are trained jointly.

128 interior points; 1–32–32–1 tanh network; 4,000 Adam + up to 200 L-BFGS iterations.

Python output: recover a field and its coefficient

A Reconstruct from 12 sensors



B Identify the coefficient

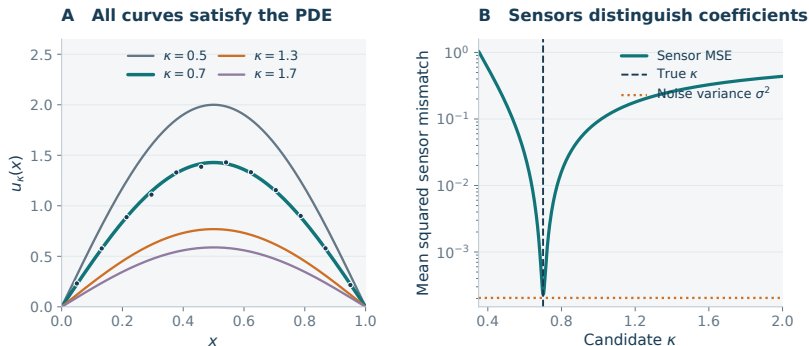


Results for this noise realization

Learned $\hat{\kappa} = \mathbf{0.700741}$ (truth 0.7); coefficient error **0.106%**. Relative field L^2 error: 3.14×10^{-3} .

The coefficient curve shows Adam iterations; the final values include L-BFGS refinement. Noise level and optimizer settings are recorded with the results.

Python output: why the observations are essential



The PDE and boundary data leave a family of solutions

For every $\kappa > 0$, $u_\kappa(x) = \sin(\pi x)/\kappa$ solves the PDE. Interior observations select a coefficient within this family.

The right panel evaluates the analytic solution family at the noisy sensors to show which coefficients fit the observations.

Section overview

From data to PDE solvers

Strong-form PINNs

Network design and boundary constraints

Variational and weak methods

Hybrid PDE-data methods

Why classical solvers remain essential

Checking a neural solution

Why classical numerical methods are still used

- ▶ *Can we reach a requested error tolerance at a predictable cost?*
- ▶ **Controlled refinement:** mesh size, polynomial degree, and time step give systematic ways to improve a discretization.
- ▶ **Efficient algebra:** sparse matrices, preconditioners, and multigrid exploit the structure of many PDEs.
- ▶ **Physical structure:** appropriate finite-volume, mixed, or compatible schemes can enforce conservation or other identities by construction.
- ▶ **Established practice:** error estimation, complex geometry handling, and extensively tested implementations.

A published benchmark

In the PDE benchmarks of Grossmann et al., PINNs did not beat FEM in solution time and accuracy on the problems tested.

Grossmann et al., IMA J. Applied Mathematics (2024).

A classical route from approximation to accuracy

For conforming FEM ($V_h \subset V$) with bounded, coercive a , Céa's lemma gives

$$\|u - u_h\|_V \leq \frac{M}{\alpha} \inf_{v_h \in V_h} \|u - v_h\|_V.$$

Here M is the continuity constant and $\alpha > 0$ the coercivity constant; u_h is the exactly solved Galerkin approximation.

- ▶ With sufficient regularity and suitable meshes, approximation estimates turn this into an explicit rate in h and polynomial degree.
- ▶ Reliable a posteriori estimators are available for many problem classes and can guide local refinement.

A finite-tolerance solve still needs an error budget

$$\|u - \tilde{u}_h\|_V \leq \underbrace{\|u - u_h\|_V}_{\text{discretization}} + \underbrace{\|u_h - \tilde{u}_h\|_V}_{\text{algebraic solve}}.$$

Nonlinear, singular, or unstable problems still require appropriate analysis.

What a PINN error bound requires

An error estimate under two assumptions

If PDE stability gives $\|u - u_\theta\|_X \leq C_{\text{stab}} \sqrt{\mathcal{J}(\theta)}$ and sampling gives $\mathcal{J}(\theta) \leq \hat{\mathcal{J}}(\theta) + \varepsilon_Q$, then

$$\|u - u_\theta\|_X \leq C_{\text{stab}} \sqrt{\hat{\mathcal{J}}(\theta) + \varepsilon_Q}.$$

The loss must include the right residual and boundary norms for this PDE.

- ▶ **Representation:** a good network may exist; training need not find it.
- ▶ **Sampling:** a useful ε_Q requires regularity and coverage, with control valid for the network selected by training.
- ▶ **Optimization:** a bound assuming small training error does not prove Adam or L-BFGS reaches it at an affordable cost.

Mishra–Molinaro (2023); De Ryck–Mishra, Acta Numerica (2024). Norms, rates, and constants depend on the PDE and formulation.

Small training loss can still hide a wrong solution

Difficulty	What can go wrong
Finite collocation	Residual peaks or thin layers fall between training points.
Multiple scales	Oscillations and sharp features can learn much more slowly than the smooth background.
Time dependence	Small early-time errors and an unbalanced space-time loss can spoil long-time predictions.
Physical constraints	A generic pointwise loss does not exactly impose cell conservation, positivity, or an entropy inequality.

Adding data does not remove these issues

Noisy or sparse observations introduce their own weighting and identifiability questions, as the inverse experiment demonstrated.

Krishnapriyan et al., NeurIPS (2021); Wang-Yu-Perdikaris, JCP (2022). These are possible failure modes, not unavoidable outcomes.

What helps PINNs, and what still needs checking

Possible improvement	Remaining check
Hard boundary constraints	Interior PDE accuracy and ansatz suitability.
Adaptive collocation	Independent residual evaluation and correct sampling weights.
Feature scaling, domain splitting	Resolution of every relevant physical scale.
Better conditioning and optimizers	Actual solution error; convergence is still problem-dependent.
Weak or variational formulations	Stability, quadrature, and the chosen test space.

Check whether the change helps

Measure the solution error before and after each change. Improvements depend on the PDE, sampling, and training setup.

De Ryck–Mishra (2024); Rathore et al. (2024).

Where neural PDE solvers can have an advantage

- ▶ **High-dimensional settings.** Neural representations and stochastic sampling avoid the exponential size of tensor grids. For PDEs with exploitable structure, this can make otherwise infeasible computations practical.
- ▶ **Inverse problems with sparse or poorly distributed data.** Jointly fit the state and unknown coefficients, using the PDE to constrain regions with few measurements. This can avoid repeated full forward solves; identifiability still requires informative observations.
- ▶ **Complex geometry.** Meshfree formulations use interior and boundary samples, reducing volume-meshing and remeshing effort for intricate or changing domains. Accurate boundary representation and enforcement remain essential.

Compare total cost at matched accuracy, including training and geometry preparation.

Examples: Han–Jentzen–E (2018); Raissi et al. (2020); Sukumar–Srivastava (2022).

Section overview

From data to PDE solvers

Strong-form PINNs

Network design and boundary constraints

Variational and weak methods

Hybrid PDE-data methods

Why classical solvers remain essential

Checking a neural solution

Formulation and error checks

What to choose and what to check

- ▶ Nondimensionalise the PDE and choose a strong, weak, or energy formulation.
- ▶ Match the activation to the required derivatives; choose penalty or hard boundary enforcement.
- ▶ Track each loss component and its parameter-gradient magnitude:

$$\widehat{\mathcal{J}}_j(\theta_k), \quad \left\| \nabla_{\theta} \widehat{\mathcal{J}}_j(\theta_k) \right\|, \quad j \in \{r, b, d\}.$$

Check the computed solution

Check boundary, conservation, and other physical diagnostics on independent points. If an exact solution is available, also monitor

$$\|u_{\theta_k} - u\|_{L^2}, \quad \|u_{\theta_k} - u\|_{H^1}, \quad \|u_{\theta_k} - u\|_{L^\infty}.$$

Common failure modes

- *What should we ask when the loss decreases but the solution still looks wrong?*

residual holes $\Rightarrow \hat{\mathcal{J}}_r \ll \mathcal{J}_r$,

boundary drift $\Rightarrow \lambda_b$ too small,

flat optimisation $\Rightarrow$ saturation/scaling,

wrong branch $\Rightarrow$ nonuniqueness or poor initialisation.

Choosing among formulations

- *Which formulation matches the regularity, data, and PDE structure in front of us?*

smooth solution + simple PDE	⇒ strong PINN,
elliptic variational problem	⇒ Deep Ritz,
low regularity or high order PDE	⇒ weak/variational PINN,
sparse observations or inverse problem	⇒ hybrid data + PDE.

References

- ▶ Raissi, Perdikaris, Karniadakis: physics-informed neural networks for forward and inverse PDE problems.
- ▶ E and Yu: the Deep Ritz method for variational problems.
- ▶ Kharazmi, Zhang, Karniadakis: variational PINNs and hp-VPINNs.